Tower of Hanoi using State Space Search

ARTIFICIAL INTELLIGENCE PRACTICAL | THIRD YEAR COMPUTER ENGINEERING

1. Title

Implementation of Tower of Hanoi Problem using State Space Search (Breadth-First Search) in Artificial Intelligence.

2. Aim

To model and solve the Tower of Hanoi problem using state space search representation and search strategies (BFS) in Python, finding the optimal sequence of moves to transfer disks from a source peg to a destination peg.

3. Theory

3.1 State Space Search in AI

A State Space Search models a problem as a graph containing:

- **State (S):** A snapshot or configuration of the problem at a given instant.
- **Initial State (S_0):** The starting configuration of the problem.
- **Goal State (S_g):** The target configuration to reach.
- **Operators / Actions (A):** Valid rules or transitions from one state to a successor state.
- **Path Cost (C):** Cost associated with reaching from S_0 to S_g.

3.2 Tower of Hanoi Formalization

The Tower of Hanoi consists of three pegs ('A', 'B', 'C') and n disks of varying sizes (numbered 1 to n, where 1 is the smallest disk).

State Representation: A state is represented as a 3-tuple of tuples: `(peg_A, peg_B, peg_C)`, where each tuple holds the disks currently on that peg from bottom (largest) to top (smallest).

- **Initial State (for $n = 3$):** `((3, 2, 1), (), ())`
- **Goal State (for $n = 3$):** `((), (), (3, 2, 1))`

3.3 Rules & Constraints

- Only one disk can be moved at a time.
- A move consists of taking the top disk from one peg and placing it on another peg.
- **Constraint:** No larger disk may be placed on top of a smaller disk at any time.

3.4 Search Strategy (BFS)

By applying Breadth-First Search (BFS) on this state space graph, we explore all configurations level-by-level, guaranteeing the discovery of the shortest path (minimum moves, which equals $2^n - 1$) from the initial state to the goal state.

4. Hardware & Software Requirements

- **Operating System:** Windows 10/11, Linux, or macOS

- **Programming Language:** Python 3.x
- **IDE / Tools:** VS Code, PyCharm, Jupyter Notebook, or Google Colab

5. Procedure / Algorithm

1. **Define State Representation:** Represent state as a 3-tuple of tuples `((A), (B), (C))`.
2. **Initialization:** Set initial state `((n, n-1, ..., 1), (), ())` and goal state `((), (), (n, n-1, ..., 1))`. Initialize a Queue with `(start_state, path)` and a set `visited` for tracking explored states.
3. **Generate Successors:** For a given state, explore moves between any two pegs ***i*** and ***j*** (***i*** ≠ ***j***). A move is valid if peg ***i*** is not empty and (peg ***j*** is empty OR top disk of ***i*** < top disk of ***j***).
4. **BFS Loop:** Pop `(current_state, path)` from Queue. If `current_state == goal_state`, return `path`. Otherwise, for each unvisited successor state, mark visited and push `(successor_state, updated_path)` to Queue.
5. **Display Output:** Print total moves, nodes expanded, and step-by-step state transitions.

6. Code Implementation with Line-by-Line Explanation

*Note: Code lines are displayed in **Blue**, and line-by-line explanations are provided directly below in **Black**.*

```
from collections import deque
```

➜ *Explanation: Imports deque (double-ended queue) from Python's standard library, which allows efficient O(1) popping operations for implementing BFS queue.*

```
def is_valid_move(from_peg, to_peg):
```

➜ *Explanation: Defines a helper function to verify whether a disk transfer from a source peg (from_peg) to a destination peg (to_peg) obeys Hanoi rules.*

```
    if not from_peg:
```

➜ *Explanation: Checks if the source peg is empty. If empty, no disk is available to move, so it returns False.*

```
        return False
```

➜ *Explanation: Returns False indicating an invalid move because the source peg contains no disks.*

```
    if not to_peg:
```

➜ *Explanation: Checks if the destination peg is currently empty. Moving a disk to an empty peg is always valid.*

```
        return True
```

➜ *Explanation: Returns True since any disk can be placed onto an empty peg.*

```
    return from_peg[-1] < to_peg[-1]
```

➜ *Explanation: Compares the top disk of from_peg (from_peg[-1]) with the top disk of to_peg (to_peg[-1]). Returns True only if the moving disk is smaller than the top disk on the target peg.*

```
def get_successors(state):
```

➜ *Explanation: Defines a function that takes the current state configuration and generates all possible valid next states.*

```
    successors = []
```

➜ *Explanation: Initializes an empty list to store generated successor state tuples along with move metadata.*

```
    pegs = ['A', 'B', 'C']
```

➜ *Explanation: Defines peg label names corresponding to indices 0, 1, and 2 for formatted output display.*

```
    for i in range(3):
```

➜ *Explanation: Outer loop iterating through all 3 pegs (0, 1, 2) treated as source pegs.*

```
        for j in range(3):
```

➜ *Explanation: Inner loop iterating through all 3 pegs (0, 1, 2) treated as target pegs.*

```
            if i != j and is_valid_move(state[i], state[j]):
```

➜ *Explanation: Ensures source and destination pegs are distinct (i != j) and validates if moving the top disk from peg i to peg j follows rules.*

```
                new_state = [list(peg) for peg in state]
```

➜ *Explanation: Converts immutable tuple state into mutable lists so disks can be popped and appended.*

```
                disk = new_state[i].pop()
```

➜ *Explanation: Removes (pops) the top disk from the source peg i.*

```
                new_state[j].append(disk)
```

➜ *Explanation: Places (appends) the removed disk onto the top of target peg j.*

```
                tuple_state = tuple(tuple(peg) for peg in new_state)
```

➜ *Explanation: Converts state back into an immutable tuple of tuples so it can be hashed and stored in set/queue.*

```
                successors.append((tuple_state, pegs[i], pegs[j], disk))
```

➜ *Explanation: Appends the new valid state configuration along with move information (source peg, target peg, disk number) to successors list.*

```
    return successors
```

➜ *Explanation: Returns the full list of valid successor states reachable from the current state.*

```
def solve_tower_of_hanoi_bfs(n_disks):
```

➜ *Explanation: Main search function that implements Breadth-First Search (BFS) to find the shortest path from start state to goal state.*

```
    start_state = (tuple(range(n_disks, 0, -1)), (), ())
```

➜ *Explanation: Constructs the initial state where all n_disks are stacked on Peg A in descending order (e.g., (3, 2, 1)), while Peg B and Peg C are empty.*

```
    goal_state = ((), (), tuple(range(n_disks, 0, -1)))
```

➜ *Explanation: Constructs the goal state where Peg A and Peg B are empty, and all n_disks are stacked on Peg C in descending order (e.g., (3, 2, 1)).*

```
    queue = deque([(start_state, [])])
```

➜ *Explanation: Initializes the BFS queue storing elements as tuples: (current_state, history_path_list). Starts with initial state and empty history.*

```
    visited = {start_state}
```

➜ *Explanation: Initializes a set to keep track of visited states, preventing duplicate node expansion and infinite loops.*

```
    nodes_expanded = 0
```

➜ *Explanation: Counter variable to keep track of total number of state nodes processed during search.*

```
    while queue:
```

➜ *Explanation: BFS main loop; continues exploring state space as long as queue has unvisited states.*

```
        current_state, history = queue.popleft()
```

➜ *Explanation: Dequeues the oldest state and its move history from the front of the queue (FIFO ordering).*

```
        nodes_expanded += 1
```

➜ *Explanation: Increments the expanded nodes count.*

```
        if current_state == goal_state:
```

➜ *Explanation: Goal test: checks if current state matches target goal state.*

```
            return history, nodes_expanded
```

➜ *Explanation: Returns solution move history and total nodes expanded as soon as goal state is reached.*

```
        for next_state, src, dest, disk in get_successors(current_state):
```
➜ *Explanation: Iterates through all valid successor states generated from current state.*

```
            if next_state not in visited:
```
➜ *Explanation: Checks if the successor state has already been visited/explored.*

```
                visited.add(next_state)
```
➜ *Explanation: Marks the successor state as visited immediately to avoid redundant processing.*

```
                move_info = f"Move Disk {disk} from Peg {src} to Peg {dest}"
```
➜ *Explanation: Formats descriptive string detailing the move transition.*

```
                queue.append((next_state, history + [(next_state, move_info)]))
```
➜ *Explanation: Enqueues successor state into BFS queue along with updated move history path.*

```
    return None, nodes_expanded
```
➜ *Explanation: Returns None if queue exhausts without finding goal (failsafe).*

```
if __name__ == '__main__':
```
➜ *Explanation: Standard Python entry point execution block.*

```
    N = 3
```
➜ *Explanation: Sets number of disks for execution (N = 3).*

```
    print(f"--- Tower of Hanoi using BFS (State Space Search) for N = {N} ---")
```
➜ *Explanation: Prints program execution header.*

```
    solution_path, total_nodes = solve_tower_of_hanoi_bfs(N)
```
➜ *Explanation: Invokes BFS search function for N = 3 disks and receives move history and expanded nodes count.*

```
    if solution_path:
```
➜ *Explanation: Checks if a valid solution path was returned.*

```
        print(f"
Optimal Solution Found in {len(solution_path)} moves!")
```
➜ *Explanation: Prints total number of moves in optimal path.*

```
        print(f"Total State Nodes Expanded: {total_nodes}
")
```
➜ *Explanation: Prints total count of expanded state nodes.*

```
        print("Initial State:", tuple(range(N, 0, -1)), "()", "()")
```
➜ *Explanation: Prints starting state configuration.*

```
        print("-" * 50)
```
➜ *Explanation: Prints separator line.*

```
        for step, (state, move) in enumerate(solution_path, start=1):
```
➜ *Explanation: Iterates over each step in solution path starting from step number 1.*

```
            print(f"Step {step}: {move}")
```
➜ *Explanation: Prints step index and move description.*

```
            print(f"        State -> Peg A: {state[0]}, Peg B: {state[1]}, Peg C:
{state[2]}")
```
➜ *Explanation: Prints complete state configuration across Pegs A, B, and C after move.*

7. Program Output

```
--- Tower of Hanoi using BFS (State Space Search) for N = 3 ---

Optimal Solution Found in 7 moves!
Total State Nodes Expanded: 27

Initial State: (3, 2, 1) () ()
------------------------------------------------
Step 1: Move Disk 1 from Peg A to Peg C
        State -> Peg A: (3, 2), Peg B: (), Peg C: (1,)
Step 2: Move Disk 2 from Peg A to Peg B
        State -> Peg A: (3,), Peg B: (2,), Peg C: (1,)
Step 3: Move Disk 1 from Peg C to Peg B
        State -> Peg A: (3,), Peg B: (2, 1), Peg C: ()
Step 4: Move Disk 3 from Peg A to Peg C
        State -> Peg A: (), Peg B: (2, 1), Peg C: (3,)
Step 5: Move Disk 1 from Peg B to Peg A
        State -> Peg A: (1,), Peg B: (2,), Peg C: (3,)
Step 6: Move Disk 2 from Peg B to Peg C
        State -> Peg A: (1,), Peg B: (), Peg C: (3, 2)
Step 7: Move Disk 1 from Peg A to Peg C
        State -> Peg A: (), Peg B: (), Peg C: (3, 2, 1)
```

8. Analysis & Evaluation

1. **State Space Size:** For n disks and 3 pegs, each disk can independently be on any of the 3 pegs. Total possible valid states = 3^n. For $n = 3$, Total States = $3^3 = 27$.
2. **Time Complexity:** Since the state space size is $O(3^n)$, graph search algorithms like BFS explore up to $O(3^n)$ states in the worst case.
3. **Space Complexity:** BFS maintains a frontier queue of unvisited states. Memory consumption grows exponentially as $O(3^n)$.
4. **Optimality:** BFS guarantees finding the shortest path (optimal number of moves). For n disks, minimum moves required = $2^n - 1$ (For $n = 3$, $2^3 - 1 = 7$ moves).

9. Conclusion

The Tower of Hanoi problem was successfully modeled as a state space graph. The Breadth-First Search (BFS) technique was implemented in Python to navigate the state space, successfully producing the optimal move path consisting of $2^n - 1$ steps.

10. Viva Questions & Answers

Q1: What is a State Space in AI?

Ans: A State Space is the set of all possible configurations/states a problem can take, connected by valid action transitions from an initial state to a goal state.

Q2: Why do we use Breadth-First Search (BFS) here instead of Depth-First Search (DFS)?

Ans: BFS explores nodes level-by-level and guarantees finding the optimal (shortest path) solution in terms of total moves. DFS can get stuck in deep paths or return non-optimal move sequences unless depth limits are set.

Q3: What is the minimum number of moves required for n disks?

Ans: The minimum number of moves required is $2^n - 1$.

Q4: What is the total number of valid states in a 3-peg Tower of Hanoi problem with n disks?

Ans: There are 3^n total valid states.

Q5: Why are Python tuples used to represent the pegs in this state representation?

Ans: Tuples are immutable and hashable, allowing them to be stored inside Python 'set' collections ('visited') to check visited states in $O(1)$ time complexity.